BALI EXPERIENCE AI

Complete Product, UX, Architecture & Engineering Specification

Discover Bali. Respect Bali. Experience Bali.

A production-oriented blueprint for direct import into an AI coding agent.

Version 1.0 • September 2026

PRODUCT TYPE

AI-powered connected tourism ecosystem for Bali: discovery, personalization, itinerary planning, booking, tickets, transport, guides, rentals, payments, live-trip assistance and responsible/culturally respectful tourism.

1. Executive Summary

Bali Experience AI is a connected tourism platform rather than a conventional OTA. It connects travellers with verified vendors, destinations, experiences, events, accommodation, guides, drivers, vehicles, tickets and transportation through a shared catalogue, booking layer and AI orchestration layer.

Technology must help travellers discover and experience Bali without encouraging behaviour that damages Balinese culture, sacred spaces, communities or the environment.

North Star Metric: Completed Quality Bali Experiences per Active Traveller.

2. Product Vision

Become the digital gateway to Bali where a traveller can discover, plan, book, pay for and navigate a personalized Bali journey through one trusted ecosystem.

```
Traveller → Preferences → AI → Experiences → Itinerary → Booking → Payment
→ Driver/Guide/Transport → Live Trip → Experience → Review → Learning
```

```
Traveller ↕ AI Experience Engine ↕ Destination / Experience / Event / Hotel
↕ Guide / Driver / Vehicle / Ticket / Community ↕ Booking / Payment / Payout
↕ Cultural Governance / Responsible Tourism
```

3. Product Principles

- Personalized: adapt to explicit preferences and observed behaviour without silently changing explicit controls.
- Connected: products, vendors, destinations, events, transport and itinerary items share stable IDs.
- Transactional: discovery connects to real availability, pricing, booking and payment.
- Trustworthy: AI never invents availability, prices, events or cultural restrictions.
- Culturally respectful: cultural rules are first-class backend data and enforced before recommendations.
- Local-impact aware: experiences can expose sustainability/community attributes.
- Safe: routing, emergency information and live-trip features prioritize safety and consent.
- Scalable: modular monolith first; extract services only when justified.
- Observable: critical transactions and AI workflows have logs, metrics, traces and auditability.

4. MVP Scope

Domain	MVP capabilities
Traveller	Onboarding, preferences, discovery, AI recommendations, itinerary, destination/experience/event detail, booking, payment,
Catalogue	Destinations, experiences, packages, events, tickets, vendor profiles, media, basic availability.
Mobility	Driver, guide and vehicle models; initial booking flows; route abstraction.
Partner	Vendor registration, verification, catalogue management, availability, bookings and revenue view.
Admin	Vendor approval, experience approval, destinations, events, bookings, cultural rules.
AI	Preference interpretation, recommendation ranking, itinerary generation and conversational assistant.
Responsible tourism	Cultural guidance, restrictions, local impact and sustainability attributes.

5. Post-MVP Scope

- Dynamic itinerary replanning using live weather, traffic and inventory.
- Real-time driver location and live trip mode.
- Advanced guide/driver matching.
- Vehicle rental marketplace.
- Semantic search and vector retrieval.
- Crowd-aware destination distribution.
- Sustainability/carbon-aware routing.

- Community impact measurement.
- Multilingual and voice AI.
- B2B APIs for hotels, DMCs, airlines, travel agencies and tourism stakeholders.

6. User Roles & Permissions

Role	Responsibility	Key permissions
TRAVELLER	Discover/book	Public catalogue, profile/preferences/trips, eligible bookings, pay, review.
VENDOR	Sell products	Own profile/products/inventory/bookings; submit for approval.
GUIDE	Guiding	Profile, availability, specialties, assignments.
DRIVER	Transport	Profile, vehicle assignments, availability, jobs.
ADMIN	Operations	Moderate vendors, catalogue, bookings, users.
SUPER_ADMIN	Governance	Full administration.
CULTURAL_REVIEWER	Culture	Review high-risk experiences; manage rules.
FINANCE_ADMIN	Finance	Payments, refunds, allocations, payouts.
OPERATIONS_ADMIN	Trip ops	Drivers, guides, live-trip exceptions, alerts.

7. Technology Stack

Layer	Technology
Mobile	React Native + Expo + TypeScript + Expo Router
State	TanStack Query + Zustand
Forms	React Hook Form + Zod
Web	Next.js + TypeScript + Tailwind CSS + shadcn/ui
Backend	NestJS + TypeScript
Database	PostgreSQL + PostGIS
ORM	Prisma
Cache/Queue	Redis + BullMQ
Search	PostgreSQL FTS initially; OpenSearch/vector later
Storage	S3-compatible object storage
Realtime	WebSocket
Observability	OpenTelemetry + Sentry + Prometheus/Grafana
Infrastructure	Docker + CI/CD; cloud-provider agnostic

8. Monorepo Structure

```
bali-experience-ai/
■■■ apps/
■   ■■■ mobile/
■   ■■■ api/
■   ■■■ admin/
■   ■■■ vendor/
■■■ packages/
■   ■■■ ui/
■   ■■■ types/
■   ■■■ validation/
■   ■■■ database/
■   ■■■ ai/
■   ■■■ config/
■■■ prisma/
■■■ docs/
■■■ scripts/
■■■ infrastructure/
■■■ docker-compose.yml
```

```
■■■ pnpm-workspace.yaml
■■■ turbo.json
```

9. Domain Model

```
User
 ■■■ TravellerPreference
 ■■■ Trip
 ■■■ Booking
 ■■■ Review
 ■■■ Guide
 ■■■ Driver
Vendor
 ■■■ Experience
 ■■■ Package
 ■■■ Vehicle
 ■■■ Booking
 ■■■ Payout
Destination
 ■■■ Experience
 ■■■ Event
 ■■■ CulturalRule
Experience
 ■■■ Vendor
 ■■■ Destination
 ■■■ CulturalRule
 ■■■ Availability
 ■■■ Booking
Trip
 ■■■ ItineraryDay
 ■■■ ItineraryItem
 ■■■ Booking
Booking
 ■■■ Payment
```

10. Database Entities

Entity	Required purpose
User	Identity, locale, roles, timestamps.
TravellerPreference	0–100 preference scores; budget, style, pace.
Vendor	Business identity, verification, contact, geo, catalogue, payout.
Destination	Name, slug, geo, region/district, crowd, sustainability, cultural sensitivity.
Experience	Vendor/destination, title, duration, price, capacity, preference scores, local impact, sustainability, geo, verification.
Package / PackageItem	Bundle of products with quantities and sequence.
Event / EventTicket	Event schedule/venue and ticket inventory.
Guide / Driver	Languages, specialties/service areas, verification, ratings.
Vehicle	Type, model, capacity, fuel/transmission, rental price, availability.
Trip / ItineraryDay / Item	Trip dates, daily activities, timing, cost, route references.
Booking / Payment	Transactional state, monetary totals, payment provider state.
RevenueAllocation / Payout	Beneficiary allocation and settlement.
CulturalRule	Entity-linked guidance/restrictions, severity, source, reviewer.
Availability	Date/time/capacity/status/inventory source.
Notification / AuditLog / ConsentLog	Communication, privileged audit trail, user consent evidence.

11. Database Rules

- Use opaque UUID/CUID-style IDs.
- Use decimal/numeric for money and explicit currency.
- Store timestamps in UTC; display using destination/trip timezone.
- Use PostGIS for geographic operations.

- Unique provider webhook event IDs and idempotency keys.
- Soft-delete where history/audit matters; never destructively delete financial records.
- Keep inventory and payment state auditable.

12. Mobile Information Architecture

```
Root
■■■ Auth
■■■ Onboarding
■■■ Main Tabs
    ■■■ Home
    ■■■ Explore
    ■■■ AI
    ■■■ My Bali
    ■■■ Profile

Detail:
Experience/[id]
Destination/[id]
Event/[id]
Guide/[id]
Driver/[id]
Rental/[id]
Booking/[id]
Payment/[id]
Trip/[id]
```

13. Onboarding Flow

```
Splash → Welcome → Language → Travel dates → Party size
→ Traveller type → Preference sliders → Budget → Pace
→ Free-text wishes → AI profile summary → Home
```

14. Preference Model

Preference	Range
Spiritual	0–100
Culture	0–100
Wellness	0–100
Adventure	0–100
Culinary	0–100
Nature	0–100
Art & Creative	0–100
Nightlife	0–100
Family	0–100
Luxury	0–100

Additional constraints: budget, travel style, pace, dates, party size, language and free-text intent.

15. Home UX

```
Header: location | notifications | profile
Hero: "How do you want to experience Bali?"
Primary CTA: CREATE MY BALI
Sections: Recommended For You | Experiences Near You | What's Happening | Cultural Tip | Active Trip
Bottom nav: Home | Explore | AI | My Bali | Profile
```

16. Experience Detail

```
Hero media
Title / rating / location / price
AI Match %
Why AI recommends this
```

```
Description
What's included
Schedule / meeting point
Cultural guidelines
Local impact
Reviews / related experiences
[BOOK NOW]
```

17. My Bali & Live Trip

```
MY BALI
Day 1
  08:00 Sunrise
  10:30 Cultural experience
  13:00 Lunch
  15:00 Wellness
  18:00 Sunset

Actions: MAP | EDIT | AI OPTIMIZE | BOOKINGS

LIVE TRIP
Next activity | ETA | Driver/guide | Booking status
Cultural guidance | Route
OPEN MAP | CONTACT | AI HELP
```

18. Cultural UI

```
<CulturalTip />
<CulturalWarning />
<CulturalRestriction />

INFO = educational guidance
WARNING = behaviour/dress/photo guidance
RESTRICTED = limited visitor access
PROHIBITED = never return as actionable recommendation
```

19. REST API

Domain	Endpoints
Auth	POST /auth/register, /auth/login, /auth/send-otp, /auth/verify-otp, /auth/refresh, /auth/logout
User	GET/PATCH /users/me; GET/PUT /users/me/preferences
Catalogue	GET /destinations, /destinations/:id, /experiences, /experiences/:id, /events, /events/:id
Search	GET /search?q=...
Trips	POST /trips; GET /trips; GET/PATCH /trips/:id
Itinerary	GET /trips/:id/itinerary; item create/update/delete/reorder
AI	POST /ai/chat; /ai/recommendations; /ai/itinerary; /ai/itinerary/optimize; /ai/route
Booking	POST /bookings/quote; POST /bookings; GET /bookings; GET /bookings/:id; POST /bookings/:id/cancel
Payment	POST /payments/create; GET /payments/:id; POST /payments/webhook; POST /payments/:id/refund
Mobility	GET /guides, /guides/:id, /drivers, /drivers/:id, /vehicles, /vehicles/:id
Culture	GET /culture/rules; reviewer/admin CRUD
Reviews/Notifications	POST /reviews; GET /reviews; GET /notifications; PATCH /notifications/:id/read

20. API Response & Errors

```
{
  "success": true,
  "data": {},
  "meta": { "requestId": "req_xxx" }
}

{
  "success": false,
  "error": {
    "code": "BOOKING_UNAVAILABLE",
```

```
    "message": "The selected experience is no longer available.",
    "details": {}
  },
  "meta": { "requestId": "req_xxx" }
}
```

Error codes: VALIDATION_ERROR, UNAUTHORIZED, FORBIDDEN, NOT_FOUND, BOOKING_UNAVAILABLE, PRICE_CHANGED, PAYMENT_FAILED, CULTURAL_RESTRICTION, CONFLICT, RATE_LIMITED, INTERNAL_ERROR.

21. Booking & Inventory

```
DISCOVERY → QUOTE → AVAILABILITY CHECK → PRICE/INVENTORY LOCK
→ BOOKING PENDING → PAYMENT → VERIFIED WEBHOOK
→ CONFIRMED → NOTIFICATION → IN PROGRESS → COMPLETED → REVIEW/PAYOUT
```

Prevent concurrent overbooking with atomic inventory updates, database transactions/locking or provider reservation mechanisms. Multi-vendor packages require component reservations plus compensation/refund strategy.

22. Payment

```
Booking
 ↓
PaymentService
 ↓
PaymentProvider abstraction
 ■■■ Card
 ■■■ Bank transfer
 ■■■ QR/QRIS
 ■■■ E-wallet
 ■■■ Future providers
 ↓
Provider webhook
 ↓
Signature verification
 ↓
Idempotency
 ↓
Payment update
 ↓
Booking confirmation
```

Never store raw card data. Payment webhooks must be authenticated and idempotent.

23. Revenue Allocation

```
Booking total
 ■■■ Vendor allocation
 ■■■ Guide allocation
 ■■■ Driver allocation
 ■■■ Platform fee
 ■■■ Tax/required charges
 ■■■ Community allocation (where configured)

RevenueRule → RevenueAllocation → Payout
```

Rules must be configurable and versioned; financial calculations must be deterministic and auditable.

24. AI Architecture

```
User
 ↓
AI Gateway
 ↓
Intent / Constraint Parser
 ↓
User Context
 ↓
Candidate Retrieval
 ■■■ catalogue
 ■■■ availability
 ■■■ events
 ■■■ geography
 ■■■ cultural rules
 ■■■ route data
```

```
 ■■■ weather/traffic where available
 ↓
Ranking → Cultural Validation → Route/Budget Optimization
 ↓
LLM Planner / Response Generator
 ↓
Schema Validation → UI
```

25. AI Non-Negotiable Rules

- Never invent vendor, event, price, availability, ticket quota, booking confirmation, route facts or cultural restrictions.
- Never transact without explicit user confirmation.
- Retrieve first; LLM receives validated platform entities.
- Validate structured output with Zod/shared schemas.
- PROHIBITED entities cannot become actionable recommendations.
- Expose uncertainty instead of guessing.
- Explain recommendation reasons.

26. Recommendation Engine

```
score =
  preferenceMatch * 0.30
+ locationMatch * 0.15
+ budgetMatch * 0.15
+ availability * 0.10
+ rating * 0.10
+ culturalCompatibility * 0.10
+ sustainability * 0.05
+ localImpact * 0.05
```

```
1000 candidates
→ geo/active/verified
→ availability
→ budget
→ cultural validation
→ ~200
→ preference ranking
→ ~20
→ secondary AI re-ranking
→ top 5
```

Keep weights configurable. Candidate generation and ranking are separate services.

27. AI Tools & Output Schema

```
searchExperiences(params)
searchDestinations(params)
searchEvents(params)
searchVendors(params)
getExperience(id)
getAvailability(entityType, entityId, dateRange)
getWeather(location, dateRange)
getRoute(origin, destination, preferences)
calculatePrice(items)
validateCulture(entityType, entityId, context)
createTrip(params)
updateTrip(tripId, changes)
createItinerary(params)
optimizeItinerary(tripId, constraints)
```

```
{
  "tripTitle": "Spiritual & Cultural Bali",
  "estimatedCost": 8750000,
  "currency": "IDR",
  "days": [
    {
      "date": "2026-10-10",
      "items": [
        {
          "entityType": "EXPERIENCE",
          "entityId": "exp_123",
          "start": "08:00",
          "durationMinutes": 120,
```

```
          "reason": "Strong culture preference match"
        }
      ]
    }
  ],
  "culturalNotes": [],
  "warnings": []
}
```

28. Cultural Governance

Cultural governance is a backend domain. Rules are curated by authorized reviewers and linked to destinations, experiences or other entities.

```
CulturalRule
- id
- entityType
- entityId
- severity
- title
- description
- doText
- dontText
- dressCode
- photographyAllowed
- source
- verified
- verifiedBy
- effectiveFrom
- effectiveTo
```

Severity: INFO, WARNING, RESTRICTED, PROHIBITED. Recommendation gate: Candidate → Cultural Validation → PASS / PASS+WARNING / BLOCK.

29. Responsible Tourism

- Local Impact Score: configurable 0–100 signal for local participation/contribution.
- Sustainability Score: configurable 0–100 score based on documented platform criteria.
- Crowd-aware alternatives when reliable crowd data exists.
- Geographic distribution to reduce concentration in a few tourism zones.
- Treat sacred/restricted locations differently from ordinary commercial attractions.
- Keep guidance concise, respectful and actionable.

30. Smart Routing

```
RouteRequest:
origin, destination, departureTime, transportMode,
costPriority, timePriority, safetyPriority, scenicPriority

RouteResult:
distanceKm, durationMinutes, estimatedCost,
trafficLevel, safetyScore, routeGeometry, rationale
```

Support fastest, cheapest, safest, scenic and recommended. Integrate routing into itinerary optimization.

31. Vendor Portal

```
Dashboard
■■■ Overview
■■■ Products
■■■ Packages
■■■ Availability
■■■ Bookings
■■■ Customers
■■■ Reviews
■■■ Revenue
■■■ Payouts
■■■ Promotions
■■■ Analytics
■■■ Profile
```

Vendor lifecycle: PENDING → UNDER_REVIEW → VERIFIED. Product lifecycle: DRAFT → SUBMITTED → APPROVED → PUBLISHED.

32. Admin Portal

```
Dashboard
Users | Vendors | Destinations | Experiences | Packages
Events | Tickets | Guides | Drivers | Vehicles
Bookings | Payments | Refunds | Reviews
Cultural Rules | AI Monitoring | Analytics | Audit Logs | Settings
```

33. Notifications & Realtime

Event	Channels
Booking confirmed	Push + in-app; optional email/WhatsApp
Payment successful	Push + in-app + email
Driver/guide assigned	Push + in-app
Activity reminder	Push
Weather/traffic alert	Push + in-app
Schedule change	Push + in-app
Cancellation/refund	Push + in-app + email

WebSocket is a delivery mechanism; PostgreSQL remains the source of truth for critical state.

34. Privacy & Consent

- Separate identity, travel, preference, location and payment-related data logically.
- Explicit location consent; background tracking is opt-in and justified.
- Store consent version/timestamp.
- Never store raw card data.
- Define retention/deletion policies before production.
- Least-privilege RBAC and audit privileged operations.
- Do not reuse precise location for unrelated personalization without appropriate consent.

35. Security

- HTTPS/TLS.
- JWT access/refresh strategy with rotation/revocation.
- RBAC and resource authorization.
- DTO/input validation and output validation.
- Rate limiting on auth, AI and costly endpoints.
- Secure headers/CORS.
- Secret manager/environment secrets.
- Signed webhooks.
- Idempotency keys.
- Audit logs.
- Secure media upload and private-by-default storage.

36. Observability

```
Metrics:
API latency, error rate, booking conversion, payment failures,
AI latency, AI tool failures, recommendation CTR, cancellation,
completion, driver/guide assignment success.

Tracing:
```

```
requestId, traceId, bookingId, paymentId, tripId
```

37. Testing Strategy

Level	Coverage
Unit	Auth, RBAC, pricing, scoring, cultural validation, itinerary constraints, route scoring.
Integration	Repositories, booking→payment, webhook→confirmation, package→allocations, AI tools→retrieval.
E2E	Register→preferences→AI→itinerary→quote→booking→payment→confirmation→trip→review.
Contract	OpenAPI, provider adapters, webhook payload validation.
Security	Authorization, rate limits, uploads, webhook signatures, secret handling.

38. Primary E2E Acceptance Test

```
1. Create traveller
2. Complete preference sliders
3. Ask AI for a 4-day Bali trip
4. Retrieve valid catalogue entities
5. Run cultural validation
6. Apply route/budget constraints
7. Generate and validate itinerary
8. Traveller accepts itinerary
9. Check availability
10. Generate quote
11. Create booking
12. Create payment intent
13. Verified webhook marks payment PAID
14. Booking becomes CONFIRMED
15. Notify vendor/guide/driver
16. Start trip
17. Display live route
18. Complete experience
19. Submit review
20. Make revenue allocation eligible
```

39. Definition of Done

- TypeScript compilation passes.
- Lint passes.
- Unit/integration tests pass.
- Critical E2E tests pass.
- Migration committed.
- OpenAPI updated.
- UI has loading/empty/error/success states.
- Authorization enforced.
- Analytics implemented where applicable.
- No hard-coded secrets.
- Documentation updated.
- Feature runs from clean checkout.

40. Environments & CI/CD

Environment	Purpose
development	Local Docker services and seeded demo data.
staging	Production-like integrations and E2E.
production	Live users and transactions.

```
git push
→ install → lint → typecheck → unit → integration
→ build → security scan → staging → E2E
→ production gate → production deploy
```

41. Seed Data

Development seed: approximately 20 destinations, 50 experiences, 20 events, 20 vendors, 10 guides, 10 drivers, 20 vehicles, 10 restaurants and 10 hotels. Mark demo records explicitly so they cannot be confused with production records.

42. Implementation Roadmap

Phase	Build	Exit condition
M01	Monorepo, infra, DB, auth, RBAC, shared packages	Clean build, migration, seed, auth tests.
M02	Catalogue, vendors, destinations, experiences, events, availability, search	Browse catalogue; vendor/admin workflows pass.
M03	AI recommendation, concierge, itinerary	Grounded AI; schema/cultural validation.
M04	Trips, itinerary editor, smart route	Create/edit/reorder/optimize trip.
M05	Booking and inventory	Transactional, concurrency-safe booking.
M06	Payment, webhooks, allocations, payout	Idempotent payment confirmation.
M07	Guide, driver, rental	Eligible provider matching/booking.
M08	Live trip, realtime, notifications, reviews	Persistent recoverable live-trip state.
M09	Governance, sustainability/local impact, analytics	Cultural rules enforced.
M10	Production hardening	Staging E2E + readiness checklist pass.

43. Coding Agent Operating Rules

- Read repository/docs before changes.
- Do not rewrite working code without reason.
- Prefer small domain changes.
- Maintain shared types.
- Avoid unnecessary dependencies.
- External providers must use adapters.
- Do not expose DB models directly as public API contracts.
- Do not give LLM unrestricted write access.
- All AI mutations pass application authorization/validation.
- Run lint/typecheck/tests after each milestone.
- Stop when critical acceptance criteria fail.

44. Master Coding Agent Prompt

```
You are the principal software engineer building Bali Experience AI.

Treat the imported Bali Experience AI Coding Agent Specification as the authoritative product and technical specification.

Implement incrementally, not as a superficial mockup.

NON-NEGOTIABLE:
- Preserve working code.
- TypeScript strict mode.
- No hard-coded secrets.
- Never fabricate availability, prices, events, payments or cultural rules.
- LLM cannot directly perform privileged DB mutations.
- Financial and booking operations must be deterministic, auditable and idempotent.
- AI outputs must be grounded in retrieved application data.
- High-risk cultural experiences require cultural governance.
- Every phase must build, lint, typecheck and test.

ORDER:
M01 Foundation
M02 Catalogue
M03 AI
M04 Trips/Routes
```

```
M05 Booking
M06 Payment
M07 Mobility
M08 Live Trip
M09 Governance/Analytics
M10 Production Hardening

For each phase:
1. Inspect repository and docs.
2. Update schema/migration.
3. Implement backend domain and API.
4. Implement authorization.
5. Add tests.
6. Implement frontend and all UI states.
7. Connect to real API.
8. Add analytics.
9. Update OpenAPI/docs.
10. Run lint, typecheck, tests and build.
11. Report files changed and test results.
12. Stop if critical acceptance criteria fail.

The result must be a runnable production-oriented repository.
```

45. Phase Execution Prompts

M01 — Foundation: Build monorepo, Docker, Prisma/PostgreSQL, shared packages, NestJS API, Expo app, admin/vendor apps, JWT, RBAC, profile/preferences, OpenAPI and tests. Acceptance: install/lint/typecheck/test/build/migration/seed/auth pass.

M02 — Catalogue: Implement Vendor, Destination, Experience, Package, Event, Ticket, Guide, Driver, Vehicle, Availability, verification/approval and public catalogue/search. Acceptance: verified vendors publish; travellers see only published products.

M03 — AI: Implement AI gateway, retrieval tools, scoring, cultural validation, structured itinerary and concierge. LLM uses retrieved entities only. Acceptance: valid IDs, grounded output and cultural checks.

M04 — Trips/Routes: Implement Trip/Itinerary and route services; edit/reorder/optimize itinerary. Acceptance: user can create, edit and route-optimize trip.

M05 — Booking: Implement quote, availability, booking state machine, inventory safeguards, cancellation and multi-vendor package handling.

M06 — Payment: Implement payment adapter, intent, webhook verification/idempotency, refunds, revenue allocation and payouts. Never store raw card data.

M07 — Mobility: Implement guide/driver matching, vehicle rental and transport booking with availability and verification.

M08 — Live Trip: Implement live trip, WebSocket, notifications, driver/guide status and consent-based location tracking.

M09 — Governance/Analytics: Implement CulturalRule, reviewer workflow, sustainability/local-impact, audit logs, consent logs and analytics.

M10 — Hardening: Security/dependency audit, performance, observability, backups, migration safety, E2E, deployment and rollback documentation.

46. Critical Edge Cases

- Experience unavailable during payment → invalidate quote and offer alternatives.
- Price changes → require new confirmation.
- Payment succeeds but webhook delayed → remain pending until verified/reconciled.
- Duplicate webhook → idempotency prevents duplicate confirmation.
- Multi-vendor component fails → compensation/refund strategy.
- Driver cancellation → replacement matching and notification.
- Weather unsuitable → propose alternatives; never silently alter paid booking.
- Cultural rule changes → revalidate and notify.
- Connectivity loss → confirmed itinerary/tickets remain available offline.

• Location denied → app continues with graceful degradation.

47. Production Readiness

Area	Required
Application	Reproducible clean build.
Database	Migration review; backup/restore tested.
Security	RBAC, secrets, webhooks, rate limits, uploads.
Payments	Sandbox E2E, retries, refunds, reconciliation.
AI	Grounding, hallucination, cultural-block and schema tests.
Booking	Concurrency and cancellation tests.
Observability	Errors, traces, metrics, alerts.
Privacy	Consent/location handling documented.
Mobile	Crash monitoring, offline tickets/itinerary, deep links.
Operations	Admin/vendor support flows.
Documentation	Architecture/API/deployment/runbooks current.

48. KPI Framework

Category	Metrics
Activation	Onboarding completion; preference completion; first AI recommendation.
Discovery	Experience views; saves; recommendation CTR.
Conversion	Booking start; payment completion; booking conversion.
Quality	Experience completion; cancellation; refund; rating.
AI	Acceptance; edit rate; latency; tool errors.
Operations	Guide/driver assignment; schedule changes.
Responsible tourism	Local-impact exposure; cultural compliance; geographic distribution.

49. Final Product Definition

Bali Experience AI is a digital tourism operating layer for Bali. It connects demand, supply, inventory, mobility, experiences, events and local participation while using AI to personalize the journey. The platform must remain grounded in real data and governed by explicit cultural and operational rules.

The intended end state is an interconnected tourism ecosystem with traveller mobile software, partner tooling, operational administration, AI orchestration, transaction infrastructure and responsible-tourism governance.

50. Minimum Development Commands

```
pnpm install
pnpm lint
pnpm typecheck
pnpm test
pnpm build

docker compose up -d

pnpm prisma migrate dev
pnpm prisma db seed
```

51. Required Documentation

```
docs/
■■■ architecture.md
■■■ database.md
■■■ api.md
■■■ authentication.md
■■■ catalogue.md
■■■ booking.md
■■■ payment.md
■■■ ai.md
■■■ recommendation.md
■■■ itinerary.md
■■■ routing.md
■■■ cultural-governance.md
■■■ privacy.md
■■■ security.md
■■■ deployment.md
■■■ operations.md
■■■ contributing.md
```

52. Final Agent Instruction

If a requirement is ambiguous, prefer the smallest implementation that preserves the domain model and extension points. Do not invent business policy. Record unresolved business decisions explicitly. Never trade transaction integrity, security or cultural governance for speed.

END OF SPECIFICATION